MAE 204 - Final Project Report

Nathan Van Utrecht

March 18, 2026

1 Report Summary

1.1 Software and Approach

My approach implements a complete control pipeline for a youBot mobile manipulator consisting of four primary steps: kinematics simulation, trajectory generation, feedforward plus feedback control, and a simulation loop.

1.1.1 Kinematics Simulation

The `NextState` function predicts the robot's configuration at the next timestep based on current state and commanded velocities using a first-order Euler integration.

- **Arm and Wheels:** The next joint and wheel angles are calculated by adding the product of their respective speeds and the timestep (Δt) to the current values.
- **Chassis Odometry:** The chassis motion is updated by first calculating the body twist $\mathcal{V}_b$ from wheel velocities using the mecanum wheel kinematics described in Section 13.4 in Modern Robotics. This twist is then transformed into a change in the world coordinates (Δq) based on the current chassis orientation before being added to the current configuration.

1.1.2 Trajectory Generation

The `TrajectoryGenerator` function is decomposed into eight distinct segments to ensure a smooth pick-and-place sequence. I utilized `mr.ScrewTrajectory` to generate smooth transitions between configurations. To accommodate the CoppeliaSim gripper dynamics, the generator inserts 63 consecutive configurations (approximately 0.625 s) with the gripper state set to 1 for grasping and 0 for releasing, ensuring the fingers have sufficient time to actuate fully.

1.1.3 Feedforward Plus Feedback Control

The `FeedbackControl` function implements a PI control law combined with feedforward reference twists to drive the end-effector error to zero. The steps are as follows:

- **Error Calculation:** The error twist X_{err} is extracted from the transformation matrix between the current and desired configurations using the matrix logarithm.
- **Control Law:** The commanded twist $\mathcal{V}$ is calculated by combining the adjoint-transformed feedforward twist $\mathcal{V}_d$ with proportional (K_p) and integral (K_i) terms based on the accumulated

error.

$$\mathcal{V}(t) = [\mathrm{Ad}_{X^{-1}X_d}]\mathcal{V}_d(t) + K_p X_{err}(t) + K_i \int_0^t X_{err}(t)dt \tag{1}$$

- **Jacobian Pseudoinverse:** To map the task-space twist $\mathcal{V}$ to joint and wheel velocities $(u, \dot{\theta})$, I computed the mobile manipulator Jacobian $J_e(\theta)$ which concatenates the base and arm Jacobians. I then applied the Moore-Penrose pseudoinverse with a specified tolerance to avoid singularities.

1.1.4 Simulation Loop

The simulation loop coordinates the entire process by first generating the full end-effector reference trajectory. It then iterates through the trajectory, calling `FeedbackControl` to get commanded speeds and `NextState` to update the robot's simulated position. The script finally logs the robot configurations and error vectors into `.csv` files for visualization in CoppeliaSim.

1.2 Surprises and Enhancements

One significant challenge encountered was handling the transition between the mobile base and the 5R arm kinematics. To address potential numerical instability near singularities (such as when the arm is fully extended), I implemented a singularity avoidance enhancement in the `FeedbackControl` function. Specifically, I applied a tolerance of 1×10^{-2} to the `np.linalg.pinv` function. This treats small singular values as zero, preventing the controller from commanding infinite velocities when the Jacobian becomes rank-deficient.

I noticed self-collisions during my initial version of the code. However, instead of explicitly ensuring that the robot avoided self-collisions, I changed the angle that the gripper held the cube at. This ensured the arm never got too close to the body for self collisions.

I also observed that using `ScrewTrajectory` resulted in more natural-looking paths compared to a linear `CartesianTrajectory`, especially during the long travel segment between the initial and final standoff positions. The explicit timing logic for the gripper was also critical. Without the 63-line buffer, hte block would consistently slip as the robot began to lift before the fingers had made contact.

2 Results

2.1 Case 1: "Best"

Controller Description & Gains: For the best case, I utilized a well-tuned **feedforward-plus-proportional (P) controller** to minimize the initial error between the actual and reference end-effector configurations. As show in Figure 1, all six components of the error twist X_{err} exhibit a clean, monotonic convergence to zero within the first five seconds, demonstrating stability with little to no overshoot. The manipulability factors in Figure 2 remain well-bounded throughout the task which indicates that the robot avoids singularities. This smooth transient response ensures a reliable grasp and high-fidelity tracking of the reference trajectory.
Video Link: `https://drive.google.com/file/d/13AmKaYoUzJ41hXvFNVyeSKssX_F0vnRq/view?usp=sharing`

Table 1: Controller and Initial Configuration: Best Case

Parameter	**Value / Matrix**
Controller Type	Feedforward + P
Proportional Gain (K_p)	$\text{diag}(1.5, 1.5, 1.5, 1.5, 1.5, 1.5)$
Integral Gain (K_i)	$\text{diag}(0, 0, 0, 0, 0, 0)$
Initial Chassis Configuration (ϕ, x, y)	$(0.25, -0.25, 0.82)$ [rad, m, m]
Initial Arm Configuration $(\theta_1, \theta_2, \theta_3, \theta_4, \theta_5)$	$(0.459, -0.227, 0.482, -1.294, 0.731)$ [rad]
Initial Wheel Angles (w_1, w_2, w_3, w_4)	$(0, 0, 0, 0)$ [rad]

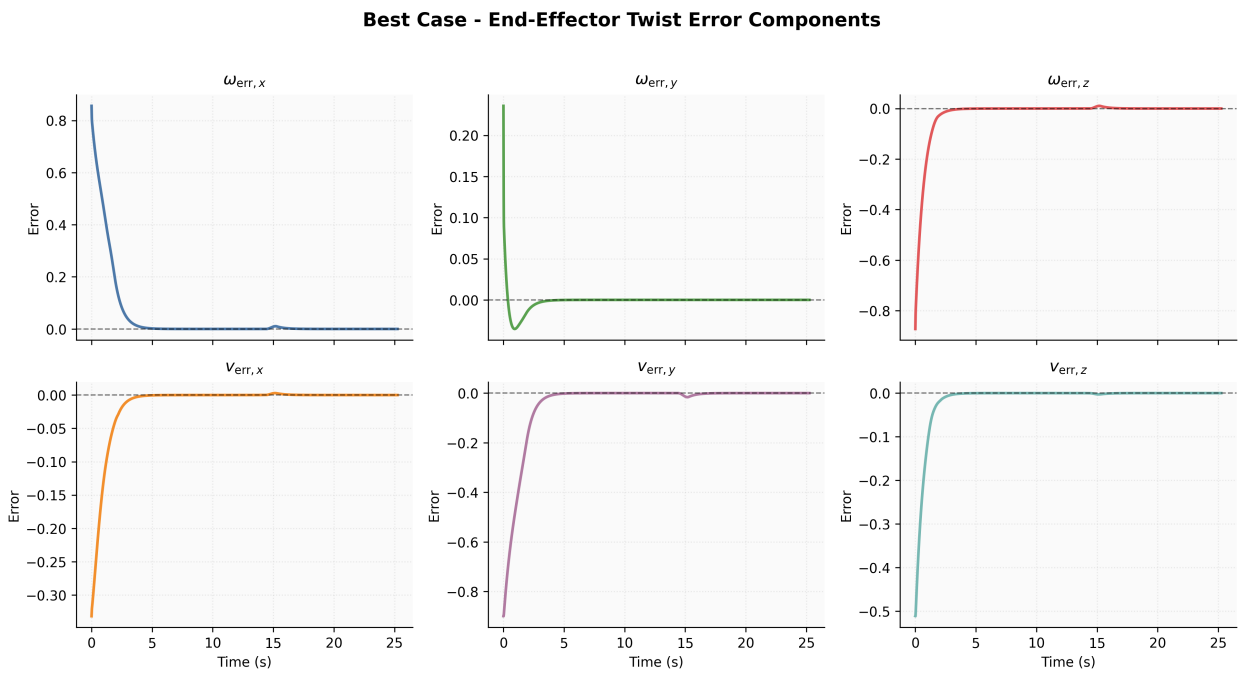

Figure 1: Plot of the six elements of X_{err} as a function of time for the best case.

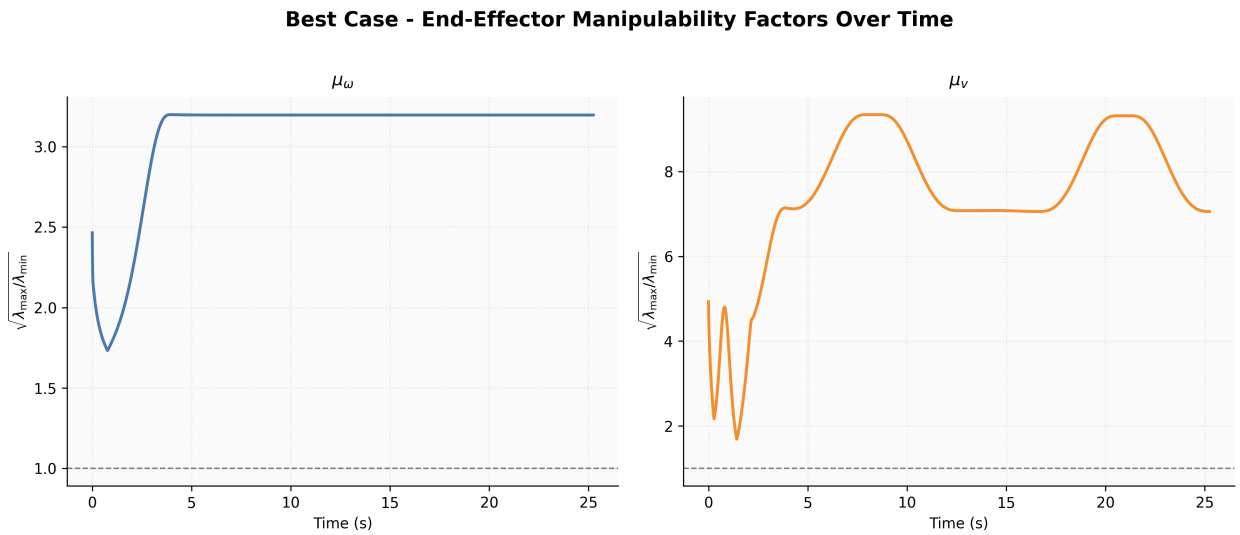

Figure 2: Plot of the manipulability factors $\mu_1(A_w)$ and $\mu_1(A_v)$ as a function of time for the best case.

2.2 Case 2: "Overshoot"

Controller Description & Gains: For the overshoot case, the controlled was configured as a **feedforward-plug-PI controller** with an increased integral gain to demonstrate non-ideal transient behavior. As observed in Figure 3, the addition of the integral term introduces a noticeable overshoot and oscillation in several error components, particularly in the angular error about the y and z axes and the linear error in the z direction. While the proportional gain drives the error down rapidly, the accumulated integral error causes the end-effector to over-correct before eventually settling to zero. Despite this oscillation, the controller successfully eliminates the initial orientation and position errors within the first five seconds, well before the end of the first trajectory segment. The manipulability factors in Figure 4 show higher variability during the initial transient phase compared to the best case, reflecting the more aggressive and oscillatory joint movements required to follow the command.

Table 3: Controller and Initial Configuration: Overshoot Case

Parameter	**Value / Matrix**
Controller Type	Feedforward + P
Proportional Gain (K_p)	$\text{diag}(4, 4, 4, 4, 4, 4)$
Integral Gain (K_i)	$\text{diag}(0.01, 0.01, 0.01, 0.01, 0.01, 0.01)$
Initial Chassis Configuration (ϕ, x, y)	$(0.25, -0.25, 0.82)$ [rad, m, m]
Initial Arm Configuration ($\theta_1, \theta_2, \theta_3, \theta_4, \theta_5$)	$(0.459, -0.227, 0.482, -1.294, 0.731)$ [rad]
Initial Wheel Angles (w_1, w_2, w_3, w_4)	$(0, 0, 0, 0)$ [rad]

Video Link: `https://drive.google.com/file/d/1sCd94aVVWE_1Vc4gZzWzNzvgGZf7IMwj/view?usp=sharing`

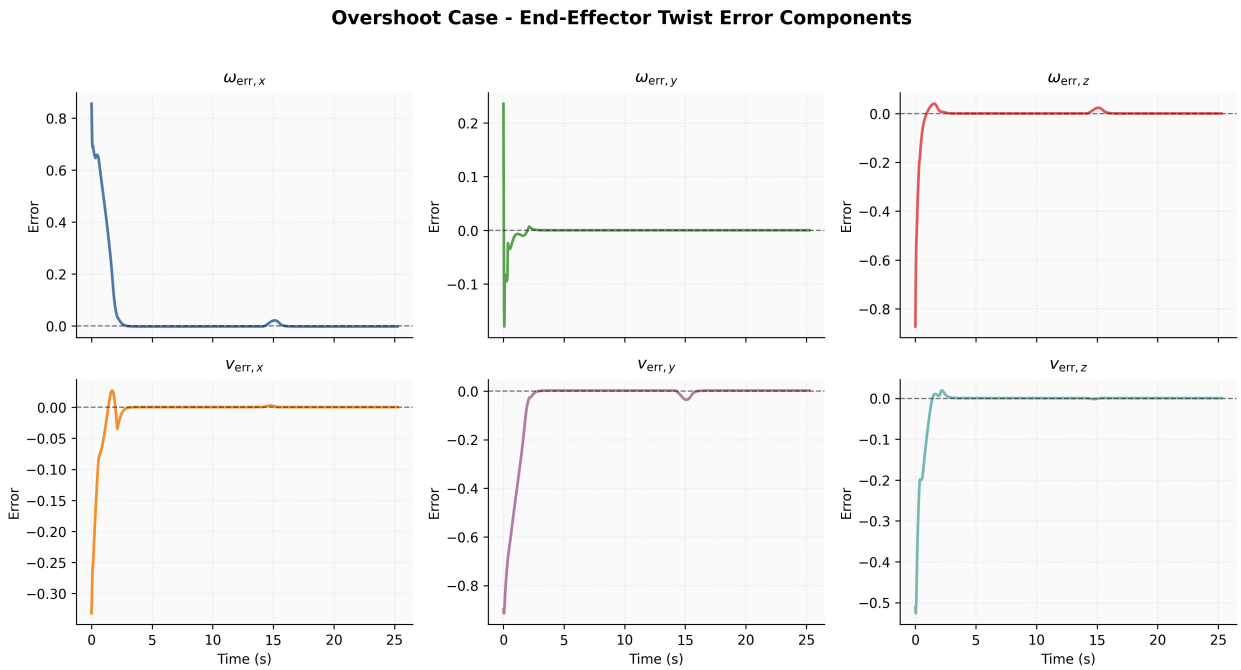

Figure 3: Plot of the six elements of X_{err} as a function of time for the overshoot case.

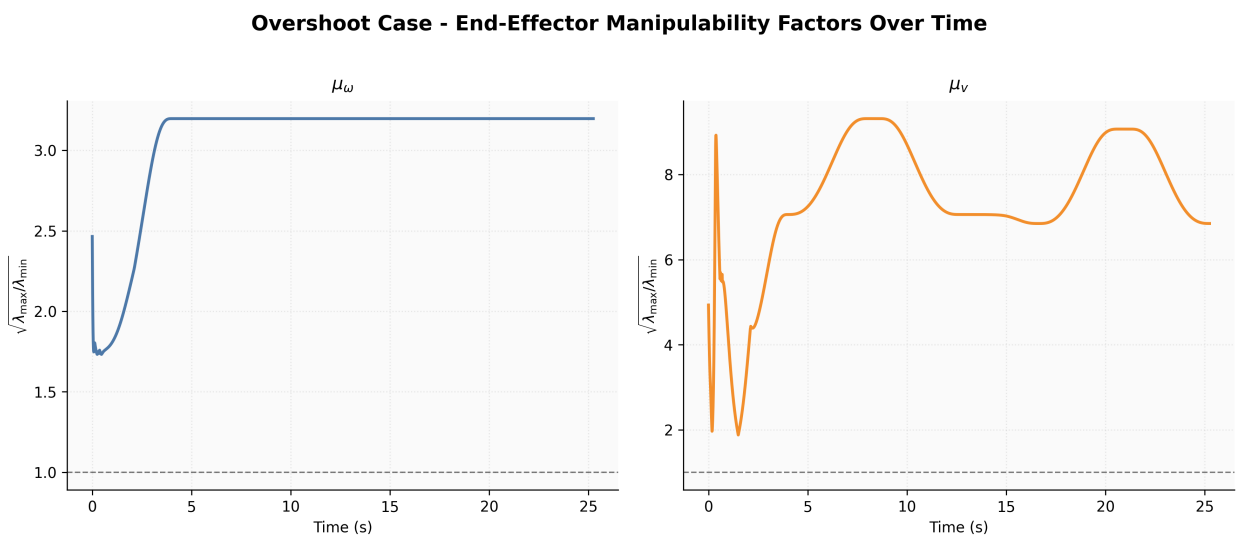

Figure 4: Plot of the manipulability factors $\mu_1(A_w)$ and $\mu_1(A_v)$ as a function of time for the overshoot case.

2.3 Case 3: "New Task"

Controller Description & Gains: The new task case demonstrates the controller's versatility in handling a custom pick-and-place scenario. For this task, the same feedforward-plus-PI controller structure to employed to maintain consistency. As shown in Figure 5, the end-effector successfully

overcomes the initial error in trajectories. The error components converge rapidly toward zero, though a distinct transient is visible around the 15-second mark as the robot transitions through the transit and placing segments of the reference trajectory. The manipulability factors in Figure 6 confirm that the chosen configurations for the new task do not drive the robot into singularities ensuring the Jacobian remains well-conditioned for velocity control throughout the motion. Table 5 shows the new task's initial and final cube configurations.

Table 4: Controller and Initial Configuration: New Task Case

Parameter	**Value / Matrix**
Controller Type	Feedforward + PI
Proportional Gain (K_p)	$\text{diag}(2, 2, 2, 2, 2, 2)$
Integral Gain (K_i)	$\text{diag}(0.01, 0.01, 0.01, 0.01, 0.01, 0.01)$
Initial Chassis Configuration (ϕ, x, y)	$(0.25, -0.25, 0.82)$ [rad, m, m]
Initial Arm Configuration $(\theta_1, \theta_2, \theta_3, \theta_4, \theta_5)$	$(0.459, -0.227, 0.482, -1.294, 0.731)$ [rad]
Initial Wheel Angles (w_1, w_2, w_3, w_4)	$(0, 0, 0, 0)$ [rad]

Table 5: Initial and Goal Cube Configurations: New Task

Configuration	**Value** (x, y, θ)
Initial Cube Configuration (ϕ, x, y)	$(0.82, 1.5, 0.5)$ [rad, m, m]
Final Cube Configuration (ϕ, x, y)	$(0, -1.0, 0.75)$ [rad, m, m]

Video Link: `https://drive.google.com/file/d/1roTeHDwBO1cRcbqnK-L8p-u8iOgbBv8y/view?usp=sharing`

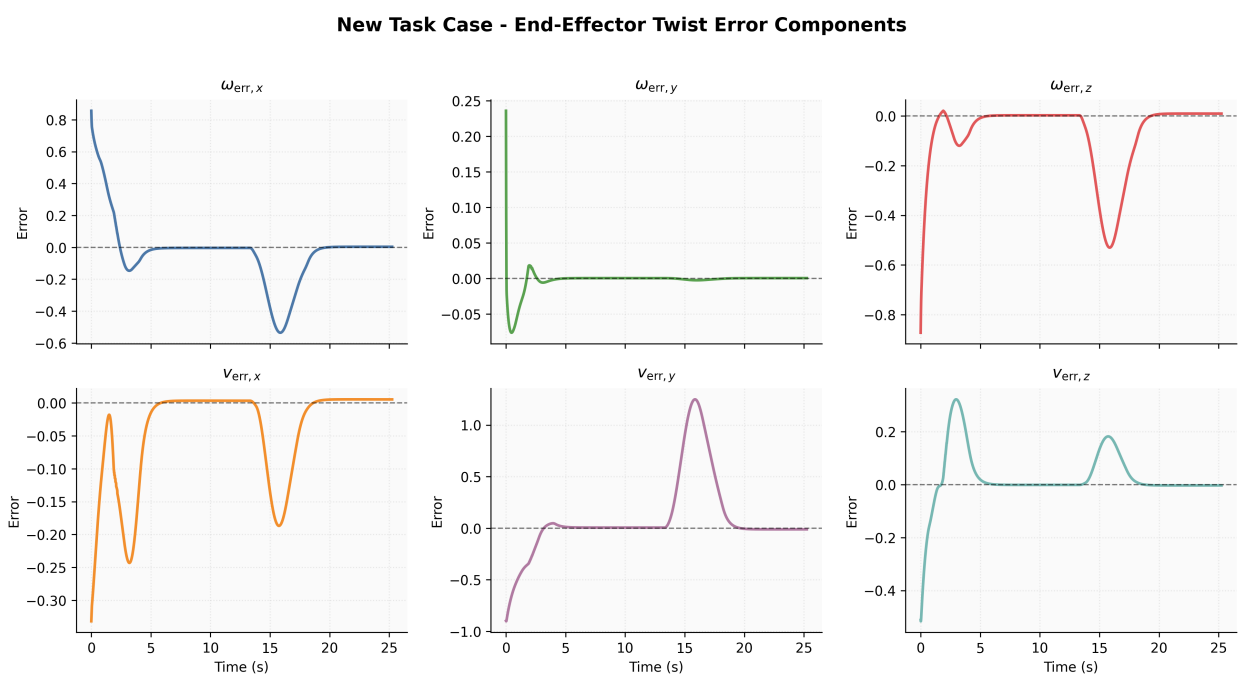

Figure 5: Plot of the six elements of X_{err} as a function of time for the new task case.

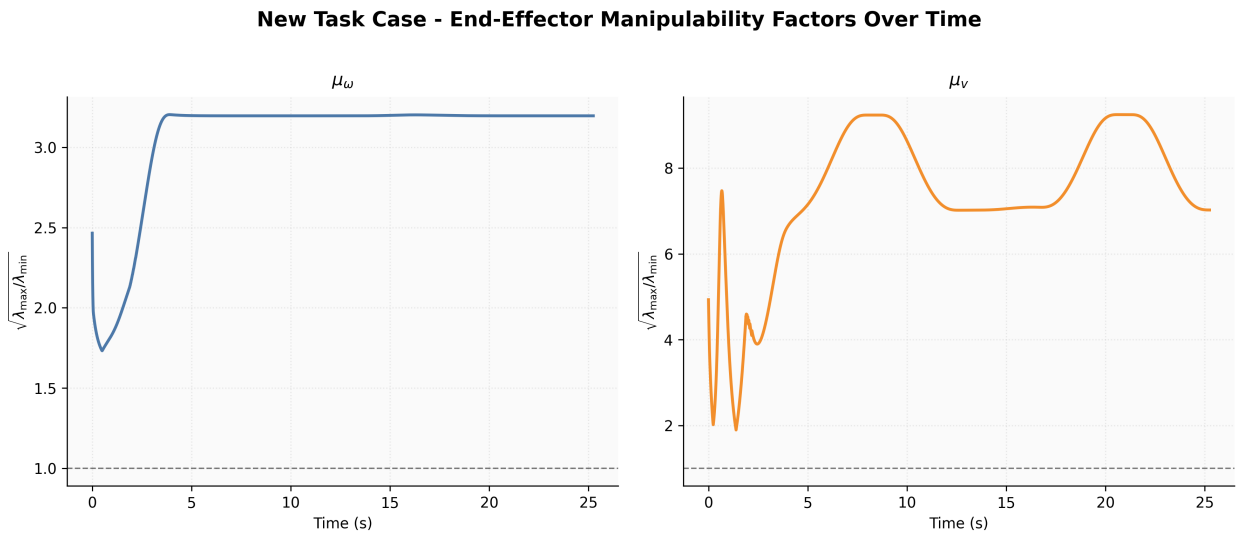

Figure 6: Plot of the manipulability factors $\mu_1(A_w)$ and $\mu_1(A_v)$ as a function of time for the new task case.

3 Discussion

1. **Integrator Term:** The primary advantage of the integrator term (K_i) is its ability to eliminate steady-state error that may rise from small numerical integration errors or persistent disturbances. However, as seen in the overshoot case error plots (Figure 3), a significant disadvantage is that the integral term continues to accumulate error while the controller is driving the end-effector toward the target. By the time the error reaches zero, the accumulated value is large, forcing the end-effector to overshoot the desired position.

2. **Manipulability Factors:** The manipulability factors represent how close the robot is to a kinematic singularity. Larger values indicate the robot is nearing a limit where it cannot move easily in certain directions. In my results, these factors became large during the transition segments where the arm was significantly extended to reach the standoff configurations. Conversely, the factors were smaller and more stable when the arm was in a more folded or neutral configuration closer to the base.

3. **Joint Velocities:** If v_{max} is reduced to a very low value, the error increases after picking up the cube because the robot can no longer achieve the commanded twist $\mathcal{V}$ required by the reference trajectory. When the calculated joint speeds exceed these low limits, they are clipped which prevents the end-effector from moving fast enough to keep up with X_d. This lag results in a growing discrepancy between the actual and desired configurations which the controller cannot correct.

4. **Velocity Control Task Examples:** Velocity control is realistic for tasks on flat surfaces where the robot's dynamics are well-modeled and the environment is controlled. However, velocity control would not be possible for tasks involving high-force interactions (i.e. heavy payload transport, tasks with lots of impact, etc.). In these scenarios, the robot must respond to dynamic forces and torques making torque or impedance control necessary.

5. **Torque Control:** To implement torque control, the `FeedbackControl` function would need additional physical parameters like the mass, inertia, and center of mass for each robot link, as well as the friction coefficients for the joints and wheels. It would also require the current joint positions and velocities as inputs to calculate the dynamic effects of gravity, Coriolis, and centrifugal forces. To compute these required torques, the software would use the `InverseDynamics` function from the Modern Robotics textbook code.

6. **Generative AI Usage:** I used AI (specifically Gemini Pro 3.1) to help structure the final project report, generate LaTeX tables for data presentation, and clean up my code after all of my debugging. While it provided assistance with formatting, I am responsible for the accuracy of the underlying control logic and verification of all generated results.